Reflexión final — TP: sistema de turnos para consultorio odontológico

Materia: Metodología I · **Integrante:** Thiago Oviedo · **Tipo:** TP individual · **Fecha:** 2026-10-15

1. Contexto y propósito

El sistema nació de un problema concreto y verificado en el research: coordinar turnos odontológicos por teléfono o WhatsApp genera ida y vuelta, dobles reservas y tiempo perdido. La propuesta es una página de turnos para un consultorio: el paciente reserva online sin llamar, y el consultorio gestiona la agenda completa (profesionales, prestaciones, bloqueos) en un solo lugar.

El alcance de v1 quedó deliberadamente chico: reserva pública sin login del paciente (con token de cancelación), agenda por profesional para el admin, estados de turno (`reservado → confirmado → completado / cancelado`) y prevención de solapamientos por profesional y por sillón. Single-tenant (un consultorio, DD-02) e integraciones externas, ficha clínica y cobros fuera de alcance desde el día uno (DD-06). No es una versión tímida de un sistema grande: es el núcleo que se puede demostrar completo.

2. Proceso metodológico seguido

El recorrido fue Discovery → Knowledge Base → roadmap → implementación, en ese orden y sin saltear pasos:

1. **Discovery (Etapa 2):** investigación de 34 sistemas en tres rondas (Argentina, LatAm, internacional) con verificación de fuentes posterior. La convención `(C)` comprobado / `(A)` declarado / `No evidenciado` fue la regla que más me sirvió: obliga a no deducir por analogía. El informe, el checklist y la bitácora quedaron en `discovery/` y `docs/`.
2. **Knowledge Base:** 10 archivos canónicos (`knowledge-base/`) con visión, actores, modelo de datos, reglas de negocio numeradas (RN-XX-NN), decisiones (DD) y supuestos (SU). Es la fuente de verdad: todo lo que se codeó después se podía rastrear hasta una sección de la KB.
3. **Roadmap:** `CHANGES.md` con 18 changes en 6 fases, árbol de dependencias, camino crítico de 10 changes y nivel de *governance* por change.
4. **Flujo OpenSpec:** `openspec init`, luego el ciclo completo explore → propose → apply → verify → archive sobre el change de consigna, y el change de fundación `c-01-foundation-setup`, cuyas 14 tasks se implementaron y archivaron el 2026-10-15 (scaffold frontend, CORS configurable y stub de seguridad).
5. **Skills:** las skills del proyecto quedaron migradas a `.agents/skills/` con su registro, para que cualquier agente (o yo, a la semana siguiente) sepa qué herramienta corresponde a cada fase.

3. Decisiones y desvíos

El desvío más importante fue **deliberado y documentado**: en vez de encadenar `C-01 → C-07` para atacar la regla de solapamientos, hice un change autónomo y acotado, `crear-turno-sin-solapamientos`, porque `C-02 core-models` y `C-03 auth-jwt-rbac` son *governance* CRITICO y no estaban archivados. Saltar esa puerta habría significado codear con la gobernanza en pausa; en cambio se declaró el desvío en la propuesta, con la trazabilidad en `CHANGES.md` para que `C-02` absorba la persistencia después. Dos ajustes más salieron de las inspecciones: acotar el change a dominio + API (Opción B, sin frontend) y **no acoplar** `crear_turno` al calendario `HorarioAtencion`, para no quedar rehén de la Pregunta 8 / SU-03 (horarios semanales fijos). También se incorporó el **sillón/box como recurso de primera clase** (DD-09): la regla RN-TU-01 se reescribió para prohibir solapamiento por profesional *y* por sillón, algo que el research no mostraba demostrado en ningún competidor.

El stack (React + Vite, FastAPI, SQLAlchemy, SQLite) fue **stack libre**, confirmado por SU-05: la cátedra no impuso tecnología ni deploy. Lo elegí por separación de capas y testabilidad (DD-01), no por moda.

Lo que quedó pendiente tiene una explicación honesta: el backend existente no salió de `C-01`, sino del change de consigna (modelos mínimos, servicio y `POST /turnos`), de modo que `C-01` quedó en deuda

hasta el 2026-10-15, cuando se implementó completo (14/14 tasks: scaffold de frontend, stub de `security.py`, CORS y `.env.example`). El mismo día también se implementó y archivó `C-02 core-models` (8 tablas, seed idempotente con hash pbkdf2 y migración Alembic `001_baseline`, con las 4 decisiones de diseño aprobadas antes de codear). El ciclo del roadmap va **3 archivados**; el que sigue en el árbol es `C-03 auth-jwt-rbac`, también con governance CRITICO.

4. Qué salió bien y qué queda pendiente

Salió bien:

- **112 tests verdes** (`backend/tests`: unitarios del servicio con casos de borde `[inicio, fin)`, colisión por profesional, por sillón, por bloqueo, integración del `409`, el reloj centralizado, la caracterización de turnos en el pasado, la fundación CORS/stub, y el modelo de datos completo con unicidades, seed e idempotencia de migraciones), y `mypy --strict` sin errores en el backend; recién ejecutados antes de escribir esto.
- **Ciclo OpenSpec completo** cerrado el 2026-10-10 con la spec sincronizada en `openspec/specs/turnos/spec.md` y el cambio archivado, más `C-01` y `C-02` archivados el 2026-10-15 (specs `foundation-setup` y `modelo-datos` sincronizadas). No quedó "código suelto": quedó spec + tests + evidencia.
- **Trazabilidad**: 8 inspecciones en `docs/bitacora-inspeccion.md`, cada una con archivo y sección que la respalda; `git log` con 18 commits con propósito claro (Discovery, etapas 5-6-7, informe, propuesta C-01, migración de skills y de docs, fundación, modelos y sus archives).
- **Honestidad documentada**: 4 URLs muertas detectadas y corregidas con registro (I-01), 0 productos inventados, vacíos de mercado dejados como `No evidenciado` en vez de rellenados.

Queda pendiente:

- Las pantallas y la lógica de la Fase 4 (`C-11` a `C-15`): existe el scaffold de React (build y `tsc` en verde), pero todavía sin páginas de reserva ni zona protegida.
- Auth JWT + RBAC (`C-03`, CRITICO, el siguiente en el árbol), motor de agenda con franjas (`C-07`), reserva pública (`C-09`), hardening de privacidad (`C-17`, CRITICO), seed demo y aceptación E2E (`C-16`, `C-18`).
- El Q&A interactivo de 11 puntos del Discovery quedó parkeado; el checklist se reconstruyó desde los raw con nota de honestidad, y la deuda quedó registrada en el state file del orquestador (`.active-orchestrator-state.json`).

5. Aprendizajes sobre ingeniería de software y trabajo individual

Repetiría: la KB antes que el código — escribir las reglas numeradas (RN-TU-01, RN-TU-02…) hizo que las decisiones de implementación fueran discusiones de una línea, no de una tarde. Repetiría el protocolo de inspección entre etapas: no es "buscar errores al final", es decidir explícitamente qué se corrige, qué se parkea y qué se documenta como pendiente; y a veces la decisión correcta es no tocar nada. Repetiría cerrar cada cambio solo con tests verdes: fue la condición objetiva de "terminado".

Corregiría: el orden. Si volviera a arrancar, ejecutaría `C-01` primero, antes que el change de consigna, para que el backend no quedara huérfano del roadmap y la trazabilidad fuera lineal. Corregiría también la planificación del Discovery: el Q&A interactivo se parkeó por falta de tiempo y obligó a reconstruir el checklist después, cuando habría sido más barato hacerlo en el momento. Y dejaría el frontend menos para el final: en el plan es el cuello de botella (5 changes secuenciales) y hoy recién existe el scaffold, con todo el valor de la Fase 4 todavía por construir.

6. Mirada metodológica

La diferencia con "solo codear" se vio en tres cosas concretas. Primero, **los specs y tests como condición de cierre**: sin ellos, "está listo" es una opinión; con ellos, es un resultado repetible por cualquiera (72 passed). Segundo, **la evidencia**: cada decisión de la bitácora cita archivo y sección, lo que permite auditar el recorrido completo desde el `git log` y los docs, en lugar de reconstruirlo de memoria.

Tercero, **los desvíos declarados por escrito**: el change autónomo no fue una improvisación, fue una decisión con su `proposal.md`, su justificación y su lugar en el roadmap.

El costo es real — escribe mucho más del que codea, y a veces la documentación corre detrás del código. Pero el beneficio también: en un TP individual, donde no hay nadie que mire mi código antes de que yo lo mire de nuevo, la metodología funciona como par de ojos externo. Lo que corregiría de raíz es la sincronización entre plan y realidad: mientras `CHANGES.md`, el state file del orquestador y lo que realmente está implementado no digan lo mismo, la trazabilidad se debilita justo en el punto donde más aporta.